

Testing and debugging functions

Project: Python readiness portfolio

1. Recall and choose your starting point

Try the recall before opening notes. Then use a different colour or a labelled second line to correct your explanation.

Reading structured data

Without opening the old lesson, explain one key idea from Reading structured data. Give an example and a mistake that the idea helps you avoid.

My remembered idea, example and correction

Return values and local state

Without opening the old lesson, explain one key idea from Return values and local state. Give an example and a mistake that the idea helps you avoid.

My remembered idea, example and correction

Loops and trace tables

Without opening the old lesson, explain one key idea from Loops and trace tables. Give an example and a mistake that the idea helps you avoid.

My remembered idea, example and correction

2. Practise with fading help

Use W2: Find a revealing case for Testing and debugging functions. Record the answer once; the lesson and workbook refer to the same attempt.

Given inputs · Procedure I chose · Middle steps · Result and check

Given inputs: _____

Procedure I chose: _____

Middle steps: _____

Result and check: _____

The step I completed without help, and the hint I needed

3. Individual reasoning check

Use the worked case for Testing and debugging functions. Proposed explanation: “A successful run proves the conclusion”. Decide whether this explanation is supported. Point to the step or supplied fact that decides; explain your repair if needed.

My explanation before partner discussion

4. Independent transfer case

Independent case: Use boundary tests to reveal a function defect

Predict the output. If the function mistakenly uses $x \leq 10$, identify the first failing assertion and whether the final print happens. State the repair and which earlier tests must be rerun. Do these cases prove all input types are handled?

```
def below_limit(x):  
    return x < 10  
  
assert below_limit(9) is True  
assert below_limit(10) is False  
assert below_limit(11) is False  
print("three checks passed")
```

My independent explanation, trace and limits

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

5. W1 Write expectations

Give expected results for `remaining(6, 2)`, `remaining(6, 6)` and `remaining(6, 0)`.

My reasoning and evidence

6. W2 Find a revealing case

A faulty function always returns 4. Which of your three cases pass and fail?

My reasoning and evidence

7. W3 Repair and retest

A faulty function adds capacity and used. For (6, 2), state actual and expected. Repair it and name a regression case.

My reasoning and evidence

8. W4 Test a new function independently

Define `stock(packs, loose)` to return `packs * 4 + loose`. Create expected tests for (2, 3), (0, 3), (2, 0), then show which detects a faulty packs + loose body.

My reasoning and evidence

9. W5 Limit the conclusion

All three stock tests pass. Does that prove correct handling of negative counts?

My reasoning and evidence

10. Project checkpoint and reflection

Add a labelled testing and debugging functions evidence sheet to Python readiness portfolio. Show how you can design revealing tests for a function. Use the supplied fictional case and keep its inputs, intermediate steps, calculation or prediction, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

My project change, supporting evidence and remaining limit

11. Feedback, return check and learning record

Evidence · My record

Evidence: Worked alone / hint / model

My record: _____

Evidence: First step I repaired and why

My record: _____

Evidence: New case checked later; date and result

My record: _____

Evidence: What I can now explain without notes

My record: _____

Evidence: One remaining question

My record: _____

Revisit this idea with your teacher after a delay. Familiarity is different from explaining and using it on a changed case.

12. Pictorial case practice: Testing and debugging functions

Use a test that catches the boundary bug. A function should classify an integer as tiny exactly when it is less than three.

Python 3 intended function: `return x<3`. Test cases: (2,True),(3,False),(4,False). Candidate bug: `return x<=3`.

Testing and debugging functions

Start with the supplied facts

Follow each arrow. Ask which fact allows the next step.



1. Record contract

Strict less-than excludes the value 3.



2. Run boundary cases

The intended outputs are True, False, False for 2,3,4.



3. Locate the bug

Using `<=` passes 2 but fails the test at 3 before the later test.

CHANGE ONE THING

Remove the test at 3 and keep only 2 and 4.

Explain the changed result using the matching lesson.

Predict the three correct outputs. Which test first catches `<=`, and why do tests at only two and four miss the bug?

My picture-based explanation and changed-case prediction

Support used: alone / hint / worked example. This record is practice evidence.

Project connection: For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted outputs and the changed-case explanation. Use actual Python 3 results to check the prediction after reasoning first.

13. Project portfolio milestone

Project: Python readiness portfolio

Current milestone: Python readiness portfolio. Keep runnable code, predictions, actual outputs, one repaired error and an explanation of a function on changed inputs.

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted outputs and the changed-case explanation. Use actual Python 3 results to check the prediction after reasoning first.

Evidence to keep · My record

Evidence to keep: This week's input and deciding step

My record: _____

Evidence to keep: Prediction and actual result (or labelled paper trace)

My record: _____

Evidence to keep: One change since the previous checkpoint and why

My record: _____

Evidence to keep: One error or limit and the next test

My record: _____

Explain what this week adds to your project. Reuse last week's record rather than starting again.

14. Supported practice and learning evidence

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Use a test that catches the boundary bug. A function should classify an integer as tiny exactly when it is less than three.

Practice 1: explain the deciding evidence

Which labelled step matches this detail? Using `<=` passes 2 but fails the test at 3 before the later test.

1. Locate the bug
2. Record contract
3. Run boundary cases

My selected conclusion, deciding fact and explanation

Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

After your attempt, compare with the supported explanation: Locate the bug Using `<=` passes 2 but fails the test at 3 before the later test. A revealing test distinguishes a correct implementation from a plausible mistake. Checking only values far from the boundary would miss this inclusive-versus-strict bug.

Practice 2: explain the deciding evidence

Changed-case practice: Remove the test at 3 and keep only 2 and 4. Which conclusion is supported by the supplied facts?

1. The buggy `<=` function passes those remaining tests.
2. This one example guarantees correct results for every future input.
3. We can decide the result without examining the changed input or the procedure.

My selected conclusion, deciding fact and explanation

Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

After your attempt, compare with the supported explanation: The buggy `<=` function passes those remaining tests. Both procedures agree away from the equality boundary.

Repair a missing building block

Review Reading structured data (python-bridge/11). Goal: Process a list of records consistently. Short practice: Circle the included rows, calculate the sum and mean, and state coverage. Repeat after adding the fourth row with count four.. Return to this week and explain what you changed.

Review Return values and local state (python-bridge/9). Goal: Use a returned value in a later calculation. Short practice: Trace the two returned values and explain why first stays seven. What happens if the caller tries to print result directly?. Return to this week and explain what you changed.

Review Loops and trace tables (python-bridge/6). Goal: Trace variable values before and after each loop iteration. Short practice: List the four visited values and their running totals. Explain why 5 is absent and what prints for `range(1,1)`.. Return to this week and explain what you changed.

Predict, run and debug

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual output or error, change one input, then run again. Keep code, predictions, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Keep your completed work

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run evidence. Open the HTML copy to read or print it. Drafts stay in this browser when storage is available; clear drafts on a shared device.

What I repaired, the evidence I kept and the next test
